

Module 5: Analyzing Data with Python

Author: Alexander Booth

Date: March 2026

Cohort: IBM-IBSC Jan 28, 2026

Overview

This module focuses on **analyzing data and building predictive models in Python**.

In earlier modules we focused on programming fundamentals, data acquisition, and working with structured data. In this module we move into the **core analytical workflow of data science**: understanding data, exploring relationships, building models, and evaluating predictions.

The workflow we follow mirrors how data scientists actually work:

1. Understand the dataset
2. Clean and preprocess the data
3. Explore relationships using statistics and visualization
4. Build predictive models
5. Evaluate and refine those models

We use the core Python data science stack throughout:

- **Pandas** for data manipulation
- **NumPy** for numerical computing
- **Matplotlib / Seaborn** for visualization
- **scikit-learn** for machine learning models

The module concludes with a **capstone analysis of housing prices in King County**, where you apply the full workflow from raw data to predictive modeling.

Data science is not just collecting data. It is about **cleaning data, understanding relationships, building models, and evaluating predictions**.

Why Data Analysis Matters

Once data is collected and loaded into your environment, the real work begins.

Real-world datasets often contain issues such as:

- Missing values
- Inconsistent formats
- Different units
- Categorical variables
- Outliers or noisy observations

Before building models, data must first be **cleaned and understood**.

After that, we try to answer questions like:

- Which variables influence the target the most?
- What patterns exist in the data?
- Can we build a model that predicts future outcomes?

Examples of predictive problems include:

- Predicting **car prices**
- Predicting **house values**
- Forecasting **sales or demand**

The overall goal is to move from:

Raw data → insight → predictive model → better decisions

1. Understanding the Data

The dataset used in this module is a **used-car pricing dataset** containing 26 attributes describing vehicles.

Examples of variables include:

- Engine size
- Horsepower
- Fuel type
- Body style
- Drive wheels
- Highway miles per gallon
- Price (target variable)

The goal of the analysis is to **predict price** using the other features.

Before doing analysis we inspect the dataset using Pandas tools:

```
.head()  
.tail()  
.info()  
.describe()
```

These help answer questions like:

- What data types exist?
- Are there missing values?
- What are the distributions of variables?

Understanding the structure of the dataset is always the **first step of analysis**.

2. Data Wrangling (Pre-processing)

Real datasets are rarely clean.

Data preprocessing transforms raw data into a format suitable for analysis.

Key tasks include:

Handling Missing Values

Missing values may appear as:

- NaN
- ?
- Blank entries

Common strategies include:

- Dropping rows with missing values
- Replacing values with the mean or median
- Replacing categorical values with the most common value

For example, if the price **is missing**, the row must be removed because it is the value we want to predict.

Data Formatting

Data from different sources may use different units or formats.

Examples:

- MPG vs L/100km
- Incorrect data types
- Inconsistent naming

Pandas functions such as:

`.astype()`
`.rename()`

help standardize data formats.

Data Normalization

Different variables may have very different ranges.

Example:

Age: 0–100
Income: 0–500,000

Normalization scales values so that variables can be compared more fairly.

Common methods include:

- Feature scaling
 - Min-max normalization
 - Z-score standardization
-

Binning

Binning groups numeric values into categories.

Example:

Car prices grouped into:

- Low
- Medium
- High

This can simplify comparisons across groups.

Converting Categorical Variables

Machine learning models require numeric inputs.

Categorical variables such as:

- fuel type
- body style
- drive wheels

must be converted into numeric form.

This is often done using **one-hot encoding**, which creates indicator variables for each category.

3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis helps us understand patterns in the data.

The key question in this module is:

Which features influence car price the most?

Descriptive Statistics

Summary statistics provide an overview of the dataset.

Important metrics include:

- Mean
- Median
- Standard deviation
- Minimum and maximum values
- Quartiles

Pandas provides tools like:

```
.describe()  
.value_counts()
```

Visualization

Visualization helps reveal patterns that statistics alone may miss.

Common plots include:

Box Plots

Show distribution, quartiles, and outliers.

Useful for comparing categories such as drive wheels vs price.

Scatter Plots

Show relationships between two variables.

Example:

Engine size vs price

If price increases as engine size increases, we observe a **positive relationship**.

GroupBy Analysis

Grouping data allows us to compare categories.

Example:

Average price by:

- drive wheels
- body style

Pivot tables and heatmaps can visualize these relationships.

Correlation

Correlation measures the strength of relationships between variables.

Values range from:

- **+1** strong positive relationship
- **0** no relationship
- **-1** strong negative relationship

Important reminder:

Correlation does not imply causation.

4. Model Development

Once we understand the data, we can begin building predictive models.

A model describes the relationship between:

- **Features (predictors)**
- **Target variable**

In this module we focus on **regression models**.

Simple Linear Regression

Simple linear regression models the relationship between one predictor and the target.

Example:

Predicting price using highway MPG.

Model form:

$$\text{price} = b_0 + b_1 * x$$

Where:

- b_0 = intercept
- b_1 = slope

Multiple Linear Regression

Multiple linear regression uses **multiple predictors**.

Example:

Predict price using:

- engine size
- horsepower
- weight
- mileage

More variables often improve prediction accuracy.

Polynomial Regression

Some relationships are not linear.

Polynomial regression adds higher-order terms (x^2 , x^3) to model curved relationships.

Pipelines

Machine learning workflows often involve multiple steps such as:

- scaling variables
- transforming features
- training models

Pipelines combine these steps into a single workflow.

5. Model Evaluation

A model that fits training data well is not necessarily a good model.

We must evaluate how well the model performs.

Visual Evaluation

Common diagnostic plots include:

- Regression plots
- Residual plots
- Distribution plots

Residual plots help detect problems such as:

- non-linear relationships
 - unequal variance
 - poor model fit
-

Numerical Evaluation

Two important metrics are used:

Mean Squared Error (MSE)

Measures the average squared difference between predicted and actual values.

Lower MSE indicates better model performance.

R-squared (R^2)

Measures how much variation in the data the model explains.

Values range from:

0 → poor fit

1 → perfect fit

6. Model Evaluation and Refinement

Models must also be tested on **new data**.

Train/Test Split

Data is divided into:

- Training set
- Testing set

Typical split:

70% training

30% testing

This estimates how the model performs on unseen data.

Cross Validation

Cross validation repeats the training/testing process across multiple data splits.

This produces a more reliable estimate of model performance.

Overfitting vs Underfitting

Two common problems:

Underfitting

The model is too simple and fails to capture patterns.

Overfitting

The model fits training data too closely and performs poorly on new data.

The goal is to find a balance between these extremes.

Ridge Regression

Ridge regression helps reduce overfitting by penalizing large model coefficients.

It introduces a parameter called **alpha** that controls model complexity.

Grid Search

Grid Search automatically tests multiple hyperparameter values to find the best model configuration.

7. Python Packages for Data Science

Several libraries support the workflow in this module.

Data manipulation

- Pandas
- NumPy

Visualization

- Matplotlib

- Seaborn

Machine learning

- scikit-learn
- Statsmodels

These libraries form the foundation of the Python data science ecosystem.

8. Final Project

The module concludes with a **housing price analysis for King County, USA**.

Students work through the full pipeline:

1. Load the dataset
2. Clean and preprocess the data
3. Perform exploratory analysis
4. Build regression models
5. Evaluate predictions

The goal is to identify which housing features most influence price and develop a linear model.

Key Takeaways

- Data preprocessing prepares datasets for analysis.
- Exploratory analysis reveals relationships and patterns.
- Regression models estimate relationships between variables.
- Evaluation metrics such as **MSE** and **R²** measure model quality.
- Train/test splits and cross-validation estimate real-world performance.
- Regularization techniques like ridge regression help prevent overfitting.

Ultimately, the data science workflow moves from:

Raw Data → Analysis → Models → Predictions → Decisions